

Where Did the Variability Go?

From Vibe Coding to Product Lines by Regeneration

Xhevahire Tërnav

LTCI, Télécom Paris, Institut Polytechnique de Paris

xhevahire.ternava@telecom-paris.fr

VARIABILITY 2026, Limassol, Cyprus

October 2, 2026

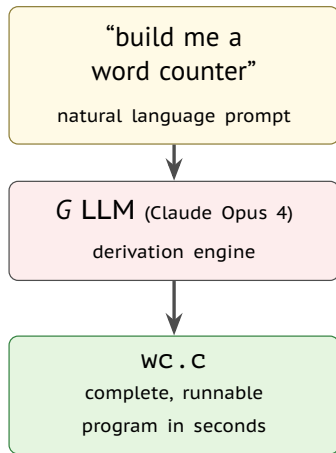
Vibe coding: a new way to write software

A new way to write software arrived last year, in which the LLM **replaces** the entire act of programming.

Vibe coding (Karpathy, 2025) works like this:

- You describe intent in **plain language**.
- The LLM writes the *entire* program.
- You **accept or regenerate**, rarely read, rarely maintain.

Practised by **15%** of professional developers (Stack Overflow, 2025), already **thousands** of vibe coded repositories on GitHub, some shipped as **real production software**.



Motivation: the variability gap

Vibe coding is new, but software variability is not. For four decades, software product lines have followed one principle: **Variability is planned, and it lives inside the artefact.**

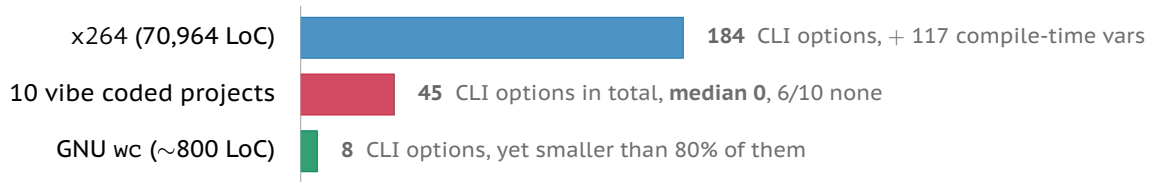
That is, variability is

- designed up front, *e.g.* in a feature model,
- implemented *in the code*, binding at a well-defined time, *e.g.* `#ifdefs` at compile time or CLI options at runtime,
- and what turns *one program* into a *configurable system*, or a whole *product family*.

Does vibe coded software retain any in-artifact variability,
or does the act of generation **eliminate it?**

Exploratory analysis: what did we find?

To answer it, we compared 10 vibe coded C/C++ projects with the configurable x264. In-artifact variability **has all but vanished**.



bar length = runtime variability (CLI options). Compile-time variability noted on the right

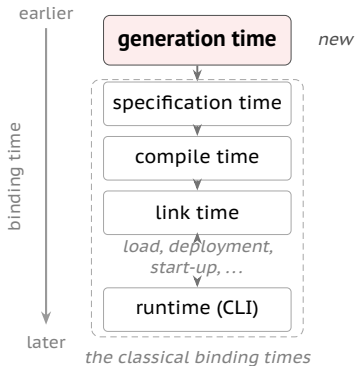
The subjects span 201 to 14,077 LoC and 7 domains. The 27 preprocessor variables are *all* incidental, **none is designed variability**. *Size does not explain the gap.*

Generation time: a new binding time

So, **where did the variability go?**

- **Traditionally**, each decision is resolved somewhere along this pipeline, and the delivered product still keeps some runtime variability.
- **In vibe coding**, the LLM resolves *every* decision at **generation time**, before any source code exists.
- **The result:** a fixed, single purpose binary, no `#ifdef`, no `getopt()`.

It did not disappear. It moved to a **new, earliest binding time**, and **into the prompt**.



Our proposal: embrace zero-variability code

Faced with this absence of variability, we see two possible reactions.

The tempting fix

Teach the LLM to emit variation points again, realised as `#ifdefs` and `getopt()` flags.

✗ generated code is already hard to understand and maintain, and adding variation points make it *even harder*.

Our proposal: embrace zero-variability code

Faced with this absence of variability, we see two possible reactions.

The tempting fix

Teach the LLM to emit variation points again, realised as `#ifdefs` and `getopt()` flags.

✗ generated code is already hard to understand and maintain, and adding variation points make it *even harder*.

Our answer

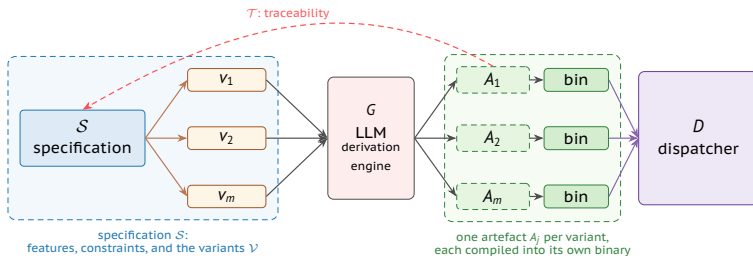
Keep every program *free* of variability. Generate many variants, dispatch the right one.

✓ variability now lives in the **specification**, not the code.

➡ **Variability by Regeneration (VbR)**

Regenerate, do not configure.

The VbR pipeline: one specification in, many variants out



VbR consists of three components:

- **Specification** $S = \langle F, \tau, \mathcal{V}, \mathcal{C} \rangle$, features, types, variants, constraints. Hand written, the only persistent artefact.
- **Generation** $G : S \times \mathcal{V} \times \Pi \times \mathcal{L} \rightarrow A$, one artefact per variant, with **no shared code base**.
- **Dispatcher** $D = \langle \mathcal{M}, \delta \rangle$, routes each request to the matching binary.

A variant is **synthesised**, not selected, as there is no pre-existing asset base.

VbR properties: three pipeline contracts

The pipeline checks three properties on *every* generated artefact:

- **Zero dead code.** Excluded features leave *no trace* in the binary, not even behind an unreachable guard.
- **Traceability.** Every included feature maps to at least one code location via `@feature` annotations.
- **Binding flexibility.** The same feature can be fixed at *generation* time in one variant, and left to *runtime* in another.

Not trust in the model, but **construction**: every artefact is *compiled and tested*, and violations are **rejected and regenerated**.

Instantiating VbR: the wc product family

We demonstrate VbR on the Unix **wc** utility, small enough to show in full, yet rich in variability. Here is one of its three variants.

S: wc.v1.yaml, the specification

```
family: wordcount
features:
  count_lines: { type: boolean }
  ... # 6 features in total
variants:
  wc-minimal: # |F| = 3 enabled
    features:
      count_lines: { enabled: true, binding: generation }
      count_words: { enabled: false }
      input_source: { value: stdin_only, binding: generation }
      cli_options: { enabled: false }
  wc-standard: ... # -l -w -c bound at runtime
  wc-analytics: ... # recursive, JSON output
```

Π: prompt template
generate_
variant.md
rules +
@feature format

prompt

G
LLM
Claude
Opus 4

generate

run once
per variant

variants/wc-minimal/wc.c → wc (12 KB)

```
/* @variant wc-minimal @spec-version 1.0 */
#include <stdio.h> /* @feature input_source.stdin_only,
                  output_format.plain_text */
/* @feature count_lines */
int main(void) {
  int c; long lines = 0;
  /* @feature count_lines, input_source.stdin_only */
  while ((c = getchar()) != EOF)
    if (c == '\n') lines++;
  /* @feature output_format.plain_text */
  printf("%ld\n", lines);
  return 0;
} /* 11 lines, 0 VP, 4 @feature */
```

@feature traceability

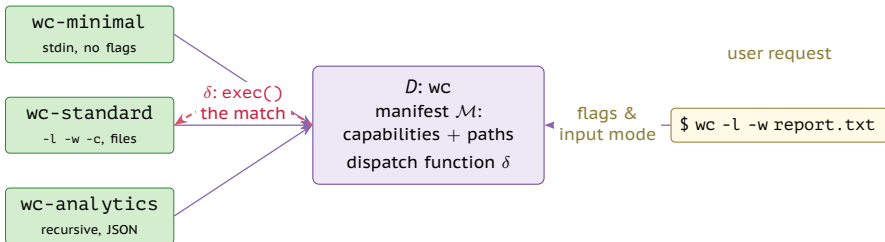
wc-standard, wc-analytics follow the same step

Variant dispatch: the user never picks a variant

The three binaries appear as a single `wc` command, and the dispatcher infers the variant from the flags and the input mode.

```
$ echo "hello world" | wc           # dispatches to wc-minimal
$ wc -l -w report.txt               # dispatches to wc-standard
```

three standalone binaries



Contrast with classical software product lines

VbR differs from classical product lines on every axis of product derivation.

	Classical SPL	VbR
Persistent asset	code base (asset base $\mathcal{A}$)	only the specification $\mathcal{S}$
Products come from	selection & composition	synthesis (regeneration)
Variation points	embedded in the code	none in the code
Binding time	consequence of mechanism	design decision per feature & variant
Reuse	shared across products	traded for small, focused binaries

In short: **classical SPL maintains code and derives products by selection**, whereas **VbR maintains only the specification and derives products by synthesis**.

Limitations: VbR is a provocation with real limits

VbR has four limits today, most of them inherited from its generator:

- **Nondeterministic generator.** Bugs or insecure idioms, no formal correctness guarantee.
- **No reuse across variants.** A fix in one does not automatically propagate.
- **One small family so far.** Scaling, cost, and consistency remain open.
- **Manual property checks.** Automating verification is needed for production use.

We mitigate: the prompt rules out dead code, and every variant must compile and pass its tests.

We plan: *anchored regeneration*, repairing a variant in place instead of rewriting it.

We accept: no code shared between variants, in exchange for smaller, focused binaries.

Conclusion: where did the variability go?

Four points to take away:

1. Vibe coding quietly moves variability **out of the code and into the prompt**.
2. **VbR** is, to our knowledge, the first product line approach for *AI generated* software.
3. It introduces a new binding time, **generation time**, with the LLM as the derivation engine.
4. Each variant is freshly generated, has **zero dead code**, and is **fully traceable**, shown on a real wc family (6 features, 3 variants).

Variability in AI generated software belongs in the specification, not in the code.

Replication package: doi.org/10.5281/zenodo.20730698